Mobilizing Transactions Transforming Business

API Integration Guide

Remote Transaction Initiation, v1.0

Jan, 2022

Document Summary:

Item	Remarks
Organization	Synergistic Financial Networks Pvt. Ltd.
Document Name	API Integration Guide
Created By	Swapnil Shelke

Document Revision and Review History:

Date	Version	Prepared By	Reviewed By	Affected Section & Summary of Change
10-Jan-2022	v1.0	Swapnil Shelke	Satyendra Singh	Initial draft

Table of Contents

1.0 – Overview 4

2.0 - Features Supported by API 4

3.0 - API Parameter Specification 4

4.0 - API Specification 5

4.1 - Device Pair API 5

4.2 - Device Pair Confirmation API 6

4.3 - Initiate Transaction 7

5.0 - Response code details 9

1.0 – Overview

This guide provides specifications of the JSON based API connectivity provided by Synergistic Financial Networks Pvt. Ltd. to its clients for integration with Notification service. This guide is intended for developers planning on integrating their systems with Notification service. This guide assumes that you are familiar with and have experience in applying the following technologies and concepts:

1. Invoking the HTTP API request using JSON payload.
2. HMAC generation.

2.0 - Features Supported by API

The features supported by this JSON API are as follows,

1. Device Pair
2. Device Pair Confirmation
3. Initiate Transaction

3.0 - API Parameter Specification

JSON API should include the following parameters as per the requirement. Any invalid parameters will be ignored or request discarded by service.

Header –

Parameter	Type	Description	Required/Optional
partnerCode	String	API client reference number	Required
hmac	String	Generated HMAC value of request payload	Required

partnerCode: Shared by Synergistic Financial Networks Pvt. Ltd with client after successful onboarding of client.

hmac: Client generate HMAC with shared secret key and request body payload.

algorithm: HmacSha512
hmac = encodeBase64(hmac('request payload', 'secret key'))

Note: Above HTTP request headers are required fields.

4.0 - API Specification

4.1 - Device Pair API

This API allow client to pair device with transaction identifier (username or terminal id).

HTTP Method: POST
API: <host>/<application>/device/v1/pair
Content Type: application/json

Request Parameter:

Parameter	Type	Description	Required/Optional
deviceSerialNo	String	Device serial number	Required
deviceIdentifier	String	Device identifier(username/tid)	Required

Response Parameter:

Parameter	Type	Description	Required/Optional
result	String	Result of request success / failed	Required
responseCode	String	Response code	Required
message	String	Error message if any	Required
confirmationCodeReference	String	Confirmation code reference code	Required

Sample Request:

```
{
   "deviceSerialNo":"14132014",
  "deviceIdentifier":"1236547890"
}
```

Sample Response:

```
{
   "result": "success",
   "message": "confirmation code published successfully.",
   "responseCode": "00",
   "confirmationCodeReference": "254"
}
```

4.2 - Device Pair Confirmation API

This API allow client to complete device pairing with confirmation code received on terminal.

HTTP Method: POST
Path: <host>/<application>/device/v1/pair/validate
Content Type: application/json

Request Parameter:

Parameter	Type	Description	Required/Optional
deviceSerialNo	String	Device serial number	Required
confirmationCode	String	Confirmation code received on terminal.	Required
confirmationCodeReference	String	Confirmation code reference received in device pair API response	Required

Response Parameter:

Parameter	Type	Description	Required/Optional
result	String	Result of request success / failed	Required
responseCode	String	Response code	Required
message	String	Error message if any	Required

Sample Request:

```
{
  "confirmationCode": "0462",
  "confirmationCodeReference": "254",
  "deviceSerialNo":"14132014"
}
```

Sample Response:

```
{
  "result": "success",
  "message": "device paired successfully.",
  "responseCode": "00"
}
```

4.3 - Initiate Transaction

This API allow client to initiate remote transaction on device(terminal).

HTTP Method: POST
Path: <host>/<application>/ transaction/v1/initiate
Content Type: application/json

Request Parameter:

Parameter	Type	Description	Required/Optional
transactionDetails	String	Encrypted request data in HEX String format	Required

Encryption Algorithm: AES/GCM/NoPadding
Encryption Key: Shared with client on successful onboarding.
Key Size: 256
GCM IV Length: 12
GCM Tag Length: 16

Parameter	Type	Description	Required/Optional
userName	String	Terminal Username	Required
password	String	Terminal user pin / password	Required
customerMobileNo	Number (10)	Customer mobile number (mandatory for WALLET payment)	Conditional
customerEmail	Email	Customer email address (required for SMSPay transaction if payment link need to send on email.)	Conditional
invoiceNo	String	Merchant reference number	Optional
transactionAmount	Decimal (8, 2)	Transaction amount	Required
additionalAmount	Decimal (8, 2)	Tip Amount (mandatory for Sale+Tip transaction)	Conditional
walletProvider	String	Wallet provider code.	Conditional

transactionType	Integer	Transaction type	Required
upiAddress	String	Customer VPA (mandatory for UPI transaction)	Conditional
payerName	String	Payer name (mandatory for cash or cheque transaction)	Conditional
chequeNo	String	Cheque No (mandatory for cheque transaction)	Conditional
autoInitiateFlag	String	Transaction auto initiate flag(Y/N), by default - N	Required
upiType	String	UPI payment generation flag (QR or VPA)	Conditional

Response Parameter:

Parameter	**Type**	**Description**	**Required/Optional**
result	String	Result of request success / failed	Required
responseCode	String	Response code	Required
message	String	Error message if any	Required

Sample Request:


```
{
  "userName": "7666297333",
  "password": "1321",
  "customerMobileNo": "9923097475",
  "invoiceNo": "123425345435",
  "transactionAmount": "1000",
  "transactionType": "22",
  "upiAddress": "9955567475@ybl",
  "payerName": "swapnil",
  "autoInitiateFlag": "N",
  "upiType": "VPA"
}
```


Encrypted Sample Request:

{ "transactionDetails":"012C03B8F2FB69DEF41DA61F977F5041B21FA20CC0B18098A2C9E90787D94B515C087B9FAB9F11D87AB701F797D976D93AEB49E0C99995717E9A0C40730421A6F631C93507

D0C711782BDF08881A7287427EA5590321492FE8A69F8BC474B102B77FA5FF0776D0D8EA474FC8
C8CF777B399160BD26ABB04E08A48C14FB9382EB0CB9DD55A84E68D6FB2D2BB6A55C7EE04B303
9DA3722F97DCC065EB5545B39BCDBEC10FEABBA72031BA950A3EABCC757BD9B4A2BB3E9303B73
7A75780A3AC93873F059D48C3F70F0E4F28C6747635B01B1E00EFCC377B376468F8CD7ED8DE2710
A56D22F8B4C695CA6AC0AA519D949656356C6"
}

Sample Response:

{
"result": "success",
"message": "Transaction details published successfully.",
"responseCode": "00",
"deviceIdentifier": "7666297333"
}

5.0 - Response code details

The possible error response will be as follows,

Response Code	Message
00	confirmation code published successfully.
00	device paired successfully.
00	device paired successfully but acknowledgement pending.
100	message authentication failed.
101	invalid request.
102	invalid partner code.
202/204/302	technical error occurred.
203	communication error between host and device.
303	device already paired with {identifier} identifier.
301/305	device not found or not active.
306	device is blocked, unblock first then try again.
307	device not active for pairing.
305	device not active for requested identifier.